

1. Explain the different memory areas managed by JVM.

Java Memory Management.

Q1) Explain the different memory areas managed by JVM.

→ Stack Memory :-

- Stack is the one of memory region which is divided by the JVM.
- A stack is created when a thread is created.
- Stack is used to store method execution data, local variables, object references, method arguments and return address.
- Each thread has its own stack to ensure thread safety.

→ Heap Memory :

- Heap is created when JVM starts
- JVM allows user to allocate memory when the "new" keyword is used the object is allocated in the heap. Its reference will be stored in stack.
- Heap memory is shared among all threads in the JVM.
- Heap memory is responsible to store the all primitive fields like int, char.....

→ Method Area:-

- Method Area also created when JVM started.
- Method Area is used to store ~~local~~ static variables, class level information, method bytecodes and constant pools.
- Method area is basically a part of the heap.

→ Program Counter:-

- Each thread has a programCounter.
- For non-native methods, it stores the address of the current JVM instruction.
- For non-native methods, PC value is undefined.

→ Native Method stacks:-

- Native method is also known as C stacks.
- It is not written in Java language.
- This memory is allocated for each thread when it is created.

2. Differentiate between Stack and Heap memory with examples.

Q2) Differentiate between stack and heap memory with examples.	
Heap	Stack
→ Stores object and instance variables. → Larger but slower → Random access. → shared among all threads → Allocation is manual by using "new". → <code>File f = new File("t.txt");</code> object should store in heap	- Stores methods calls and local variables. - smaller but faster. - last in first out. → Each thread has its own stack. → Allocation automatically. <code>File f = new File("t.txt");</code> reference "f" is stored in stack,

3. What is Garbage Collection? How does it work in Java?

Q3) What is Garbage Collection? How does it work?	
• Garbage Collection in Java is an automatic memory management process that clears memory which is occupied by unwanted objects. • It is key feature of Java virtual machine, that eliminates the need of manual memory deallocation.	• It • gar • Wha • all • Ser • loc • sy

Working:

- Identifies which object are used referred and which are unused reference.
- Garbage collection removes the unused objects.
- It removes the objects that are unreachable.
- Garbage collection is responsible to deallocate memory for unused and unreachable objects.

4. What are the different types of Garbage Collectors available in Java? Mention any two.

Q4) What are the different types of Garbage collectors available in java? mention any two.

→ Serial Garbage Collector:

- Serial Garbage collector is the simplest type of garbage collector.
- It uses a single thread to handle all garbage collection tasks.
- When garbage collection is triggered, it pauses all running application threads.
- Serial Garbage collector is used where low memory required like embedded systems.

→ Parallel Garbage Collector:

- Parallel Garbage collector is default garbage collector in Java 8.
- It is also known as throughput collector.
- It uses multiple threads to perform garbage collection.
- Parallel Garbage collector is better than single garbage collector.
- It is designed to maximize the performance.

5. What is the role of the finalize() method in Java? Is it recommended to use it? Why/Why not?

Q5) What is the role of finalize() method in Java? Is it recommended to use it? Why/why not?

- finalize() is method of the Object class in Java.
- finalize is called by the Garbage Collector before deallocating memory of an object.
- It is used to closing files, database connections, cleaning memory.
- finalize() method ~~when~~ not recommended to use due to deprecated, performance overhead.